

Ink Narrative Validator: User Manual & Technical Reference

1. Introduction

The Ink Narrative Validator is an analysis tool for interactive narratives written in the Ink scripting language (Inkle Studios). It consumes compiled Ink JSON files, constructs an internal Narrative Flow Graph (NFG) representation [1], and runs three kinds of validation to identify authoring errors in a story.

What it detects:

- Unreachable nodes: knots that no player path can ever visit.
- Dead ends: knots from which no terminal (ending) can be reached.
- State contradictions: variable-state conflicts that permanently block transitions.

The tool is grounded in Verbrugge's [1] formal definition of the Narrative Flow Graph, with some changes to better suit Ink and Visual Novels.

2. Installation

2.1. Requirements

- Python 3.10 or higher
- pip (comes with Python)

2.2. Clone the Repository

```
git clone https://github.com/n-kirstyy/ink-narrative-validator.git
```

```
cd ink-narrative-validator
```

2.3. Install Dependencies

Install all required Python packages with:

```
pip install psutil matplotlib pandas
```

2.4. Prepare JSON Files

Stories must be compiled to JSON using either the Inky editor (File → Export to JSON) or the command-line inklecate compiler: `inklecate -o story.json story.ink`

3. Usage

3.1. Validate a Folder of Stories

The primary entry point is `validator.py`. Pass it a folder containing one or more compiled Ink JSON files:

```
python validator.py /path/to/your/json/folder
```

If no path is supplied, you will be prompted to enter one.

3.2. Output Location

All output is written to a directory called `validation_output/` created in your working directory:

Path	Contents
<code>validation_output/results.csv</code>	Full numerical results for all validators and files

validation_output/reports/	Human-readable .txt report per story
validation_output/plots/	Performance plots (runtime, memory, CPU) as PNG images
validation_output/story_graphs/	Narrative structure graph image per story

4. The Narrative Flow Graph (NFG)

Module: NFG.py

4.1. Components

Component	Description
Vertices (V)	All narrative states (knots in Ink)
Hyperedges (E)	Transitions between states (are flattened to directed edges that carry condition and assignment metadata in this implementation)
Axiom (a)	The designated start node (the START knot in Ink)
Terminals (w / ℓ)	Designated ending nodes (marked by end/done in Ink); here $w = \ell$ (Simple NFG)
Context (C)	Subset of tail nodes required but not consumed by a transition (variables checked and not edited in this implementation)

4.2. Hyperedge Structure

Each Hyperedge object models one Ink choice branch and carries:

- Knot the transition originates from
- Knot the transition leads to
- List of (variable, required_bool) pairs that must hold for the transition to be available
- Dictionary of variable assignments made when the transition is taken

- Conditions that are checked but not consumed (derived automatically)

4.3. Terminal Types

- Knots with an explicit end/done token and no outgoing diverts.
- Knots with an end/done token AND outgoing diverts. These are valid author-acknowledged stopping points, but are not true terminals.

5. Validation Modules

5.1. Unreachable Node Detection

Module: unreachable_node_validation.py

Detects vertices that have no path from the axiom. A knot is unreachable when no sequence of player choices can ever reach it, making the content inaccessible.

Output Fields

Field	Description
Detected	True if any unreachable nodes exist
Reachable Count	Number of nodes reachable from the axiom
Unreachable Count	Number of unreachable nodes
Unreachable Nodes	Sorted list of unreachable node names
Runtime	Wall-clock time in seconds
Memory Peak (kb)	Peak memory usage in kilobytes
Cpu Percent	CPU usage percentage

5.2. Dead End Detection

Module: dead_end_validation.py

Detects vertices that are reachable from the axiom but from which no terminal node can ever be reached. A dead-end knot traps the player in a branch with no way to finish the story.

Output Fields

Field	Description
Detected	True if any dead-end nodes exist
Reachable from Start	Number of nodes reachable from the axiom
Can Reach End	Number of nodes that can reach at least one terminal
Dead End Count	Number of dead-end nodes
Dead End Nodes	Sorted list of dead-end node names
Runtime / Memory / CPU	Performance metrics (same as above)

5.3. State Contradiction Detection

Module: contradiction_validation.py

Detects conflicts arising from the Boolean variable world state. Unlike structural issues, state contradictions occur when a path is structurally valid, but variable assignments along it make a required transition permanently unavailable.

Output Fields

Field	Description
Detected	True if any contradiction sub-check finds an issue
Unsatisfiable Edge Count	Number of edges with contradictory conditions
Unsatisfiable Edges	List of {source, dest, variable} dicts
Dead Branch Count	Number of dead branches detected
Dead Branches	List of {node, dest, variable, required, actual} dicts
State Dead End Count	Number of state dead-end nodes
State Dead Ends	Sorted list of state dead-end node names

Conditionally Unreachable	Sorted list of conditionally unreachable node names
Runtime / Memory / CPU	Performance metrics

6. Ink Parser

Module: ink_parser.py

The parser reads a compiled Ink JSON file and constructs an NFG object. It handles the following aspects of Ink's compiled format:

- Knot extraction: identifies all top-level narrative sections (knots) as NFG vertices.
- Terminal identification: detects end/done tokens; distinguishes hard terminals (no outgoing diverts) from soft terminals (end token with optional outgoing paths).
- Hyperedge extraction: parses choice clusters (c-N keys), ev/.../ev condition blocks, VAR? condition reads, and VAR= assignment writes.
- Axiom detection: always the START knot; raises ValueError if absent.
- Divert resolution: cleans raw divert destinations, filtering out Ink-internal targets (done, end) and relative paths.

The parser is designed to be simple for the purposes of this thesis.

7. Graph Constructor

Module: graph_constructor.py

Converts an NFG object into a NetworkX directed graph and generates a PNG visualisation of the narrative structure saved to validation_output/story_graphs/. Node colours indicate role (green = axiom, red = terminal, yellow = soft terminal, light blue = regular state), and edge styles

indicate transition type (solid dark = unconditional, purple = conditional, dashed orange = context relation).

Where multiple transitions connect the same source-destination pair, the graph stores only one edge. The visualisation also becomes harder to read as node count increases, and layout improvements are a further area for future work.

8. Author Reports

For each story file, validator.py writes a plain-text report to `validation_output/reports/<story>_report.txt`.

These reports are designed to be read by story authors without understanding the underlying graph model or other project details. The test runs and plotting provided are unnecessary to the authors and were used for the study of the Thesis. Only this output and graph visualisation would be necessary on a finished product aimed at authors.

Each report contains three sections:

- Dead Ends – lists all dead-end nodes, with state-caused ones marked with *.
- Unreachable Nodes – lists all unreachable nodes, with state-caused ones marked with *.
- Contradictions – lists unsatisfiable edges and dead branches with variable details.

Example entry:

```
Contradictions - 1
-----
Dead branches (variable state conflict on path):
  gatehouse_barrier -> monastery_exit [variable: has_rope requires true, was set to false]
```

9. Performance Measurement (For Purposes of this Study)

Each validator is run 10 times per story file. The mean, standard deviation, and variance of runtime, memory peak, and CPU usage are recorded in results.csv.

Performance plots are generated for:

- Unreachable node detection: BFS vs DFS, against node count (or branching factor).
- Dead end detection: BFS vs DFS, against node count (or branching factor).
- State contradiction detection: single curve, against node count (or branching factor).
- Total validation cost: all validators summed, against node count.

To plot against branching factor instead of node count, set `PLOT_BY_BRANCHING_FACTOR = True` in `validator.py`.

10. Module Reference

Module	Responsibility
NFG.py	NFG and Hyperedge data classes
ink_parser.py	Parses Ink JSON into an NFG object
traversal_algorithms.py	BFS/DFS forward and reverse traversals
unreachable_node_validation.py	Detects structurally unreachable nodes
dead_end_validation.py	Detects structural dead-end nodes
contradiction_validation.py	Detects state-based contradictions, dead branches, and condition-caused unreachable nodes and dead ends
validator.py	Runs all validators, writes CSV, generates plots, and author reports

11. Known Limitations

- **VERY IMPORTANT:** The parser was designed to be simple and able to extract complete NFGs for testing, and not all ink structures were fully explored. Among the design decisions made, it was written to ignore global variable declarations. Therefore, contradiction detection needs a variable to be assigned along a path in the story before it is added to the state key, as it will ignore states set during initial global declarations. While a change to initially declare variables at the START node was considered, it was ultimately dropped, since it would keep variables in the state key even if their first use occurs further down the story.
- Only compiled Ink JSON files are supported. Raw .ink source files must be compiled first.
- The variable model is limited to boolean (true/false) variables.
- Story graphs in the current version become harder to read as node counts increase.
- Performance on very variable-dense stories may be slow for contradiction detection. The full extent of this remains unexplored.
- The validator detects structural issues, not semantic ones, existing in the writing itself, e.g. plot holes, typos, etc.

References

- [1] Clark Verbrugge. 2003. A Structure for Modern Computer Narratives. In *Computers and Games*, 2003. Springer, Berlin, Heidelberg, 308–325. https://doi.org/10.1007/978-3-540-40031-8_21